SQL 처리기 구현 계획서

C 언어 기반의 파일형 DB SQL 처리기를 구현한다. 목표는 텍스트 파일로 작성된 SQL을 CLI로 입력받아 파싱하고, 그 결과를 바탕으로 파일 기반 저장소에서 INSERT와 SELECT를 실행하는 것이다. 이번 구현은 빠르게 동작하는 결과물과 팀원이 단계별로 코드를 이해하고 설명할 수 있는 구조를 동시에 만족하도록 설계한다.

1. 전체 구현 원칙

실행 흐름은 아래 순서로 고정한다.

`CLI 입력 -> SQL 파일 읽기 -> 파싱 -> 실행 -> 파일 저장/조회 -> 결과 출력`

필수 구현 범위

- CLI 입력 처리
- INSERT 지원
- SELECT 지원
- CSV 저장
- 스키마 검증

코드 작성 원칙

- 함수는 하나의 역할만 맡도록 작게 분리
- 복잡한 추상화와 트릭성 구현 지양
- 에러 메시지는 파싱/실행 단계로 구분
- 팀원이 설명 가능한 수준의 구조 유지

`WHERE`, `JOIN`, `UPDATE`, `DELETE`, `CREATE TABLE`은 1차 구현에서 제외한다.

2. 저장 포맷과 스키마 규칙

저장 포맷 선택

- **CSV**, **binary**, **custom text format**을 비교한 뒤 최종 저장 포맷은 **CSV**로 선택한다.
- **binary**는 사람이 직접 확인하기 어렵고 디버깅이 불편하다.
- **custom format**은 포맷 규칙과 전용 파서를 추가 설계해야 해서 하루 과제 범위에 비해 복잡하다.
- **CSV**는 메모장으로 열어 바로 검증할 수 있고, INSERT와 SELECT 흐름 설명이 가장 쉽다.

CSV 규칙

- 각 테이블은 `data/<table_name>.csv` 파일 하나로 관리한다.
- 첫 줄은 헤더이며 컬럼명 목록을 저장한다.
- 구분자는 쉼표 `,` 를 사용한다.
- 쉼표와 큰따옴표가 없는 문자열은 그대로 저장한다.
- 쉼표나 큰따옴표가 포함되면 전체 값을 큰따옴표로 감싼다.

- 값 내부 큰따옴표는 `""` 로 escape 한다.
- 빈 문자열은 `""` 로 저장한다.
- 1차 구현에서는 값 내부 줄바꿈은 허용하지 않는다.

테이블 및 스키마 존재 규칙

- `schema/<table_name>.meta` 와 `data/<table_name>.csv` 가 모두 존재해야 유효한 테이블이다.
- `meta` 파일에는 테이블명과 컬럼 순서를 저장한다.
- CSV 첫 줄 헤더는 `meta` 파일 컬럼 순서와 완전히 같아야 한다.
- INSERT와 SELECT 시 요청 컬럼은 반드시 스키마에 존재해야 한다.
- 부분 컬럼 INSERT는 허용하며, 누락된 컬럼은 빈 문자열로 저장한다.

```
table=users
columns=id,name,age
```

3. 권장 구현 순서

1. **Foundation**: 프로젝트 구조, 빌드 방식, 실행 명령 `sqlparser <sql-file-path>` 확정, SQL 파일 읽기
2. **Schema/Storage**: 테이블 존재 규칙, meta 파일 형식, CSV 저장 규칙 확정
3. **Parser**: INSERT / SELECT 최소 문법 파싱, 토큰 분리, AST 구조 정의
4. **Execution**: 스키마 검증, INSERT 실행, SELECT 실행 연결
5. **Quality**: 파싱 에러/실행 에러 정리, 단위 테스트, 기능 테스트 작성
6. **README/Demo**: README를 발표자료 기준으로 정리하고 실행 예시와 검증 내용을 포함

이 순서는 그대로 코드 이해 순서이자 발표 설명 순서가 된다. 먼저 입출력과 저장 규칙을 안정화하고, 그다음 문법 해석, 마지막에 실행과 검증을 붙이는 방식으로 리스크를 줄인다.

4. 브랜치 생성 규칙

- 브랜치는 학습용 단계 단위로 생성한다.
- 각 브랜치는 반드시 이전 단계 브랜치에서 생성한다.
- 즉, 각 브랜치는 이전 단계까지의 모든 내용과 현재 단계 내용을 함께 포함하는 누적형 브랜치다.
- 브랜치 이름 형식은 `step/번호-주제` 로 통일한다.
- `main` 은 최종 통합본만 유지한다.

권장 브랜치 흐름

1. `step/01-foundation` : 프로젝트 구조, 빌드, CLI 실행 틀, SQL 파일 읽기
2. `step/02-schema-storage` : `step/01-foundation` 에서 생성, 스키마 규칙과 CSV 저장 규칙 추가
3. `step/03-parser` : `step/02-schema-storage` 에서 생성, INSERT/SELECT 파싱과 AST 추가

4. `step/04-execution` : `step/03-parser` 에서 생성, 스키마 검증과 실행 로직 추가
5. `step/05-quality` : `step/04-execution` 에서 생성, 에러 처리와 테스트 추가
6. `step/06-readme-demo` : `step/05-quality` 에서 생성, README와 데모 자료 정리

팀원은 브랜치를 앞에서부터 순서대로 보면서 기능이 어떻게 쌓이는지 자연스럽게 이해할 수 있다.

5. 핵심 인터페이스와 데이터 구조

- **Token**: 토큰 종류, 원문 문자열, 위치 정보
- **InsertStatement**: 테이블명, 컬럼 목록, 값 목록
- **SelectStatement**: 테이블명, 조회 컬럼 목록, 전체 조회 여부
- **Statement**: INSERT / SELECT 구분용 tagged union
- **Schema**: 테이블명, 컬럼 목록, 컬럼 수

```
char *read_sql_file(const char *path);
TokenArray lex(const char *input);
ParseResult parse_statement(TokenArray *tokens);
SchemaResult load_schema(const char *schema_dir, const char *table_name, Schema *out);
ExecResult execute_statement(const Statement *stmt, const char *data_dir, const char
*schema_dir);
StorageResult append_row_csv(const Schema *schema, const char *table_name, const Row *row);
StorageResult read_table_csv(const Schema *schema, const char *table_name, TableData *out);
```

6. 테스트 계획

단위 테스트

- lexer: 키워드, 식별자, 문자열, 숫자, 쉼표, 괄호, 세미콜론 분리 검증
- parser: INSERT / SELECT AST 생성, 괄호 누락, 세미콜론 누락, 컬럼 수와 값 수 불일치 검증
- schema: meta 파일 파싱, meta 와 CSV 헤더 일치, 없는 테이블/스키마 검증
- storage: CSV 헤더 읽기/쓰기, 쉼표 포함 문자열, 큰따옴표 escape, 빈 문자열 저장 검증
- executor: INSERT 후 파일 반영, SELECT 전체 조회와 특정 컬럼 조회 검증

기능 테스트

- SQL 파일 입력으로 INSERT에서 SELECT까지 전체 플로우 검증
- 부분 컬럼 INSERT 후 비지정 컬럼이 빈 문자열로 저장되는지 검증
- 없는 테이블, 없는 컬럼, 잘못된 SQL, 빈 데이터 파일 처리 검증
- CSV 특수문자 포함 문자열 처리 검증
- 값 내부 줄바꿈 입력 시 에러 반환 검증

7. 품질 기준과 발표 준비

- 핵심 파싱 함수, 스키마 로딩 함수, CSV 저장 함수는 반드시 단위 테스트를 가진다.
- 기능 테스트는 실제 CLI 실행 기준으로 작성한다.
- 정상 케이스뿐 아니라 실패 케이스를 반드시 포함한다.
- README에는 실행법, 디렉터리 구조, 스키마 규칙, CSV 규칙, 지원 SQL, 테스트 방법, 데모 흐름을 포함한다.
- 발표는 README 기준으로 진행하므로, 최종 단계에서 README를 사실상 발표자료처럼 정리한다.

> 기본 디렉터리 구조는 `schema/` 와 `data/` 를 사용한다. `schema/<table>.meta` 와 `data/<table>.csv` 가 모두 있어야 유효한 테이블로 본다. 1차 구현에서는 멀티 SQL 실행, 고급 SQL 문법, 값 내부 줄바꿈은 다루지 않는다.